

Automation: Workflows (Foundation)

Power CAT

Module 3: Workflows

Overview

In [Module 1](#), you built **Order Automation** — a cloud flow that routes new orders through approval and creates an invoice. In [Module 2](#), you fed it orders from a legacy file. Eventually those orders ship, arrive, and close — and that's when customers start asking a new question: *can I return this?* Answering it means checking the order's status, weighing its shipping fee, and getting a manager's sign-off above certain values — today a slow, manual detour. In this module, you build **Validate Returns**: a workflow that a customer-facing agent calls as a tool, mid-conversation, and gets a grounded answer back — built on the same Orders data every module in this series shares.

A workflow is Copilot Studio's automation format: a visual designer with native AI actions. It's the pattern that makes agents reliable — instead of leaving a decision to the model alone, the agent calls a workflow where **deterministic branches decide the business outcome** (whether the order is eligible for return, and which approval tier its shipping fee requires), **generative AI only drafts the customer- and approver-facing wording**, and **a person signs off** on higher-value cases. This module is built in the new workflows experience (see [Prerequisites](#)).

Learning objectives

By the end of this module, you will:

- Create a workflow in Copilot Studio's new workflow experience, triggered by an agent.
- Read Dataverse data with **List rows** and OData filters.
- Use Copilot's expression builder to generate flow expressions from plain English.
- Branch logic with **If/Else** on a business rule (order status).
- Generate customer- and approver-facing messages with the **M365 Copilot** action.
- Route a numeric value into approval tiers with deterministic **If/Else** conditions.
- Add **Human review** approvals routed to Outlook.
- Test the flow from the designer, and consume it from a Copilot Studio agent.

Prerequisites

The scenario continues the earlier modules, but the build stands alone — you can complete this module without them. You need:

- A **Microsoft Copilot Studio license** in your environment. For the current licensing, workflow availability, and how to obtain access, see the official [Copilot Studio licensing documentation](#) rather than assuming a specific commercial offer.
- The **new workflows experience** available in your environment. It's in public preview, so labels and layouts may shift slightly as it evolves — if a control is named differently on your screen, look for the closest match.

- A security role that lets you **read and edit the Orders table** in Dataverse — Step 8 stages test data by changing order records — and the ability to **receive and respond to human-review requests** at your Outlook mailbox.
- The **Northwind Traders** solution installed and seeded with sample data, including the **Orders** table and its model-driven apps — Step 8 uses the **Admin Management App** to stage test data. Follow the import and **Northwind Sample Data** steps in [solutions/README.md](#). Seeded orders arrive with empty statuses, so Step 8 also shows you how to stage the closed orders the return rule depends on — the module stands alone either way.
- Access to **Outlook** email with the same account, so human-review approvals can be delivered to you.
- Awareness that **published workflows consume Copilot Studio capacity when they run** — though workflows triggered by **When an agent calls the workflow** are **no charge for Microsoft 365 Copilot-licensed users**, and **all test runs in this lab are free**. Capacity and Copilot Credit metering change over time, so check the official [billing and capacity documentation](#) for current terms, and see [Appendix A: Credit consumption & cost transparency](#) for the full picture. Forecast usage with the [agent usage estimator](#).
- Plan to **clean up test data** afterward: Step 8 changes the **status and shipping fee** of four shared Northwind sample orders. Note their original values before editing so you can restore them, or reseed the Northwind data afterward (see [solutions/README.md](#)).

Business use case

Scenario: Customers ask whether an order can be returned. Determining eligibility means checking the order status, evaluating the shipping fee, and — above certain thresholds — obtaining manager or senior-manager approval. Today that's a slow, manual process, and a returns agent has to leave the conversation to work it out.

Solution: An agent calls the **Validate Returns** workflow, which confirms the order is closed, evaluates the shipping fee against approval tiers, generates the right customer or approver message, and requests human sign-off when needed — all from a single conversation turn. The agent relays the result back to the customer in plain language.

The return rule is grounded in the series' own data. Each order moves through the statuses **New → Invoiced → Shipped → Closed**, and a return is only allowed once the order reaches **Closed** (status value 3) — the point at which the customer has received it and its lifecycle is complete. For those closed orders, the **shipping fee** then decides the approval path: higher fees require a manager or senior manager to sign off before the return proceeds.

Names used in this lab

As you build, you'll rename the workflow and each action to the names this module uses. The instructions introduce every name at the step where you create it, and later steps and dynamic-content pickers refer to those exact names — so rename as you go and your screen will keep matching the instructions.

Step 1: Create the workflow

1. Go to <https://copilotstudio.microsoft.com>.



Setup check: Copilot Studio opens in whichever experience you used last, so you may land in the **classic** portal — you can tell because its left navigation shows **Flows** rather than **Workflows**. Switch to the new experience with the **Try now** button on the **New Copilot Studio experience** banner across the top of the home page — it's easy to overlook.

The classic Copilot Studio home page with the New Copilot Studio experience banner and its Try now button highlighted

Figure: The Try now button on the classic home page banner — the switch into the new experience.

 **Setup check:** Once you're in the new experience, select the environment picker in the **bottom-left corner** and confirm you're in the environment where you built the earlier modules, with the Northwind Traders solution installed.

2. In the left navigation, select **Workflows**, then choose **New workflow**. This opens the workflow designer directly — there is no create dialog and no solution picker in the new experience.

The Copilot Studio home page where you start a new workflow

Figure: The Copilot Studio home page where you start a new workflow.

3. In the designer, open the **Trigger type** and change it to **When an agent calls the workflow**. This is what makes the workflow available to agents as a tool.

The workflow designer with the Trigger type control highlighted on the trigger node

Figure: Where to change the Trigger type in the designer.

4. Rename the workflow: select the title **Untitled workflow** and enter:

Validate Returns

5. Notice that adding the **When an agent calls the workflow** trigger automatically adds a **Respond to the agent** action at the end of the flow — the paired action that returns a result into the conversation.

The renamed Validate Returns workflow with the trigger set to When an agent calls the workflow and the Respond to the agent action added automatically

Figure: The trigger set to “When an agent calls the workflow”, the Respond to the agent action added automatically — and the workflow already renamed to Validate Returns.

 **Checkpoint:** Your draft workflow is named **Validate Returns** and shows two nodes: **When an agent calls the workflow** → **Respond to the agent**.

Step 2: Read the order data

The agent passes an order number; the flow reads the matching order from Dataverse so every later decision is grounded in trusted data.

1. On the **When an agent calls the workflow** trigger, select **Add an input** and add a **Text** input named:

Order Number

Adding the Order Number input the agent passes to the flow

Figure: Adding the Order Number input the agent will pass to the flow.

2. Add a new action after the trigger — the new designer lays the flow out horizontally, so it appears to the **right** of the trigger, not beneath it. Search for **Dataverse List rows** and select the **List rows** action.

The Add action dialog with “list rows” searched, showing the Microsoft Dataverse List rows action

Figure: Adding the Microsoft Dataverse **List rows** action from the Add dialog.

3. Confirm the **Connection** is set correctly, then set **Table name** to **Orders**. If prompted, create the Microsoft Dataverse connection first.

Creating a Microsoft Dataverse connection with a display name and OAuth authentication

Figure: Creating the Microsoft Dataverse connection the List rows action uses.

The List rows Configure panel with Table name set to Orders

Figure: Setting the List rows **Table name** to **Orders**.

4. You now need the logical name of the **Order Number** column to filter on. In a new tab, go to <https://make.powerapps.com> and open the **Orders** table, then open its **Columns** page and locate the **Order Number** column. Select it, choose **Edit**, expand **Advanced options**, and copy the **Logical name**.

 **Setup check:** Power Apps opens in its own environment context — check the environment picker in the upper-right corner and switch to the same environment as your workflow before looking for the Orders table.

 **Tip:** In Northwind Traders the **Order Number** column’s logical name is typically `nwind_ordernumber`. Always copy the exact value from your own environment rather than assuming it.

5. Back in the flow, set **Filter rows** to match the copied column against the **Order Number** input, using the Dataverse **eq** (equals) operator:

```
nwind_ordernumber eq '{@triggerBody()?['text']}
```

6. Set **Row count** to **1** so the action returns only a single order.

7. Rename the action to make the flow easier to read:

```
List Order Rows
```

8. Select **Save** so none of your progress is lost.

The List rows action filtered on the Order Number column with a row count of 1

Figure: The List rows action filtered on the **Order Number** column with a row count of 1.

 **Checkpoint:** The flow reads a single **Order** that matches the **Order Number** passed by the agent.

Step 3: Branch on the return-eligibility rule

A return is only allowed when the order is closed. You capture the reply in a variable, then branch on the order’s status. In the Northwind **Orders Status** choice column the values are `New = 0`, `Invoiced =`

1, Shipped = 2, and Closed = 3, so a closed order has **order status 3** — that’s the value this branch tests.

1. Add a **Variable** action to hold the response you’ll send back — the node lands on the canvas named **Variable**. In its panel, under **Initialize Variable**, set **Variable name** to `response`, set **Type** to **String**, and leave **Value** empty.
2. The action’s own title stays **Variable** — rename it so the flow reads at a glance, the same way you renamed **List Order Rows**. Select the **Variable** title at the top of the panel and enter:

```
Initialize Response
```

 **Tip:** The same panel can hold more than one variable operation — the **+ Initialize Variable** and **+ Update Variable** buttons at the bottom add more operations to that panel. This lab needs only the single `response` variable here.

The Variable action panel initializing a String variable named response

Figure: Initializing the String variable named response, with the action renamed to Initialize Response.

3. Add an **If/Else** (condition) action to check whether the order status is closed.
4. In the condition’s left value, choose **Insert expression** and use the Copilot expression builder. Enter this natural-language prompt to generate the expression:

```
Get the first row of List Order Rows action and get the order status ID value
```

The condition’s left value with the Insert expression control highlighted, opening the Copilot expression builder

Figure: Where to find Insert expression on the condition’s left value — the way into the Copilot expression builder.

5. Insert the generated expression. It resolves the order-status value from the **first** returned row of **List Order Rows** — conceptually, `first(<List Order Rows output>['nwind_orderstatusid'])`. The expression builder references the List rows action by an internal connector ID rather than its display name, so the exact text on your screen will differ from anyone else’s — that’s expected. Don’t edit or tidy the generated reference: confirm only that it takes the **first row** and reads the `nwind_orderstatusid` value, then accept it as generated.
6. Set the operator to **is not equal to** and the value to **3**. Rename the branch so it reads **Order is not closed**. The **Else** branch handles closed orders — the ones eligible for a return.
 **Empty status:** An order whose status is **empty** also takes the **Order is not closed** branch, because an empty value is “not equal to 3” — an order that hasn’t reached **Closed** yet isn’t returnable, which is exactly the behavior you want.

The If/Else condition comparing the order status against the closed value 3

Figure: The **Order is not closed** condition comparing order status against the closed value (3).

 **Harden it later:** This build assumes the order number matches a real order. To also handle an order number that matches *no* rows — so a later expression that reads “the first row” can’t fail on an empty result — add the optional guard in [Step 10 \(Optional\): Handle an order that doesn’t exist](#).

✓ **Checkpoint:** An **If/Else** condition splits the flow: the **Order is not closed** branch handles orders that aren't closed yet, and the **Else** branch continues for closed, return-eligible orders.

Step 4: Draft and return the not-eligible message

For an order that isn't closed, the flow explains — in customer-ready language — why the return can't proceed, and returns that message to the agent.

1. In the **Order is not closed** branch, add an **M365 Copilot** action and connect it from the branch. Create its connection if prompted, then rename the action:

Draft Not Eligible Message

2. In the M365 Copilot **Message**, enter the instruction:

Create a short customer-ready message that the return for order [Order Number] is not eligible because the order is not yet closed. An order is closed only once the customer has received it. Write a single ready-to-send message with no placeholders. Use only the supplied order number and eligibility result; do not invent policy details, dates, or reasons, and do not include citations.

After typing the prompt, replace [Order Number] with the **Order Number** dynamic value **from the trigger** — in the dynamic-content picker, take it from the **When an agent calls the workflow** group, not from **List Order Rows**. Both groups offer an order number: the trigger's is a single value; the List rows one is a per-row column, and picking it makes the designer wrap your action in a loop.

⚠ **Important:** If an **Apply to each** container suddenly appears around an action after you insert a dynamic value, you picked the per-row value — the inserted chip reads `item()...`, and the flow checker will report broken references. To recover: remove the action from the loop, delete the **Apply to each**, restore the connections (branch → action → **Respond to the agent**), and re-insert the value from the trigger group instead.

3. Under **Advanced parameters**, leave **Prefer Async** set to **false** so the action returns an immediate, synchronous result. You can leave **Time Zone** unset — this message doesn't use any dates or times.

Configuring the M365 Copilot action to draft the not eligible message

Figure: Configuring the M365 Copilot action to draft the “not eligible” message.

4. Now wire this branch to return its message. This rewires the default connection, so it helps to know how the designer handles connections: to **insert** an action between two nodes, hover the connector arrow and select **+**; to **remove** a connection, select it and delete it; to **reconnect**, drag from a node's output dot to the target node. With that in mind:
 1. **Remove the default connection.** By default, **Respond to the agent** is connected straight from the **If/Else** — select that connection and delete it.
 2. **Add the update action.** On the branch, add a **Variable** action with **Update Variable**, rename it **Update Not Eligible Response**, and set **response** to the **Response** dynamic value of **Draft Not Eligible Message** — the generated message itself, not **Body** (the whole result object) or the citation fields.

The Update Not Eligible Response action set to Set variable on response, with the Draft Not Eligible Message Response as its value

Figure: The Update Variable action writing **Draft Not Eligible Message's Response** into the `response` variable.

1. **Reconnect the path.** Wire **Draft Not Eligible Message** → **Update Not Eligible Response** → **Respond to the agent**.
2. **Bind the output once.** In the **Respond to the agent** action, add a **Text** output named `response` and set its value to the **response variable** — take it from the **Variables** group in the picker, not from the M365 action. This binding is set once and never changes: every branch you build from here ends by writing its message into that one variable, which is what lets a single respond action serve the whole flow.

The Respond to the agent action with a response output bound to the response variable from the Variables group in the dynamic content picker

Figure: Binding the **Respond to the agent** output to the `response` variable (from the **Variables** group, not the M365 action).

1. **Save and publish.** Select **Save**, then **Publish** — publishing is required before you can run the flow, even for a *local test* (running the flow yourself from the designer with the **Run** node, before it's connected to an agent).

The finished branch with Draft Not Eligible Message connected to Respond to the agent, and the Save and Publish buttons highlighted

Figure: The finished branch, saved and published — publishing is what makes the flow runnable.

5. Test this path from the designer with the **Run** node (or the **play** button on a step): provide an **Order Number** for an order that isn't closed, and run it to validate the schema and see the returned JSON.

 **Tip:** Need an order that isn't closed? In the Power Apps tab from Step 2, open **Tables > Orders**, find the **Order Status** column, and filter it to **Does not equal > Closed** — any **Order Number** in the filtered list will do.

 **You may see `AgentTriggerTest.notTestable` (Bad Gateway).** Because this workflow's trigger is **When an agent calls the workflow**, the designer can't always invoke it directly from the **Run/play** button — it's built to run when an *agent* calls it. If you hit this error, it isn't a mistake in your build; you'll fully exercise the flow through an agent in Step 9.

The Bad Gateway error dialog reading error: `AgentTriggerTest.notTestable` when running the agent-triggered workflow directly

Figure: The failure case — running an agent-triggered workflow directly can return

`AgentTriggerTest.notTestable` ; it's fully testable through an agent in Step 9.

When the direct run *does* proceed, the Activity view completes down the not-eligible path:

The Activity view after a successful test run, with every node on the not-eligible path completed

Figure: A successful direct run — the whole not-eligible path completed in the Activity view.

 **Checkpoint:** For an order that isn't closed, the flow generates a customer-ready “not eligible” message and returns it to the agent.

Step 5: Decide the approval tier by shipping fee

For closed orders, the approval path depends on the shipping fee — and because that's an exact numeric rule, you decide it with **deterministic conditions**, not an AI classifier. You compare the fee against two thresholds: a fee **above \$50** needs senior-manager sign-off, **\$10–\$50** needs manager sign-off, and **below \$10** is approved automatically. Reusing the **If/Else** condition from Step 3 keeps the decision predictable and easy to audit.

 **Cost note:** Deterministic conditions also keep tier routing on the flat per-action billing rate and off the premium-token meter that a generative model can incur. See [Appendix A](#).

1. Testing switched the designer to the **Activity** tab — switch back to the **Build** tab so you can edit the flow again. Then, from the **Else** (order is not closed) branch, add an **If/Else** (condition) action and rename as **Senior Manager Approval Validation**.

A new If/Else condition added on the Else branch of Order is not closed, showing Needs setup

Figure: Adding the first condition on the **Else** (closed order) branch.

2. In the Senior Manager Approval condition's left value, choose **Insert expression** — the same control you used in Step 3 — and use the Copilot expression builder to generate the shipping-fee value. Enter a prompt such as:

```
Get the shipping fee from the first row of List Order Rows
```

Using the first row avoids creating a loop over the returned records. As in Step 3, accept the generated expression as-is rather than editing its internal reference.

The Expression assistant with the shipping-fee prompt, Generate, and Insert highlighted

Figure: Generating the shipping-fee expression with the Copilot Expression assistant.

3. Set the operator to **is greater than** and the value to **50**. This first split separates the senior-manager tier from everything else:
 - The **If (true)** branch is the **above \$50** tier — it needs **senior-manager** approval. Rename the if branch as **Greater than 50**.
 - The **Else (false)** branch continues to a second check.

The Senior Manager Approval Validation condition with the Greater than 50 branch set to first > 50

Figure: The senior-manager split — shipping fee **Greater than 50**.

4. On the Senior Manager Approval **Else (false)** branch, add a **second If/Else** and rename as **Manager Approval Validation**. For its left value, insert the **same** shipping-fee expression, set the operator to **is greater than or equal to**, and set the value to **10**:
 - The Manager Approval **If (true)** branch is the **\$10–\$50** tier — it needs **manager** approval. Rename the if branch as **Less than 50, Greater than 10**.
 - The Manager Approval **Else (false)** branch is the **below \$10** tier — it's **approved automatically**, with no review.

The Manager Approval Validation condition on the Else branch with Greater/equal 10, and its Less than 50, Greater than 10 branch

Figure: The manager split on the Else branch — shipping fee **Greater or equal to 10**.

5. Note how the boundaries resolve, so there's no ambiguity at exactly \$10 or \$50:

- Fee > 50 → senior manager (for example, \$50.01 and up).
- Fee ≥ 10 and ≤ 50 → manager (a fee of exactly \$10 or exactly \$50 lands here).
- Fee < 10 → approved automatically.

💡 **Why not the Classify action?** Classify uses an AI model to sort an input into categories — the right tool for genuinely *semantic* text (for example, routing an email by its intent). But for exact currency thresholds, a deterministic numeric comparison is more reliable and predictable: it always resolves the same way, and its behavior at the boundaries is defined by you, not inferred by a model. The completed shipping-fee decision — the two nested conditions and their three resulting branches:

The nested Senior Manager and Manager Approval Validation conditions producing three branches: Greater than 50, Less than 50 Greater than 10, and Else

Figure: The deterministic shipping-fee decision — **Greater than 50** (senior), **Less than 50, Greater than 10** (manager), and the **Else** below-\$10 approved branch.

✅ **Checkpoint:** The flow now has three shipping-fee outcomes, each on a named branch you created in this step:

- **Greater than 50** — the **senior-manager** branch (fee above \$50).
- **Less than 50, Greater than 10** — the **manager** branch (fee \$10–\$50).
- The **Else** of *Manager Approval Validation* — the **approved** branch (fee below \$10, no review).

The next steps refer to these as the **senior-manager**, **manager**, and **approved** branches — the names you just gave them on the canvas.

Step 6: Generate messages and add human review

Each tier gets its own generated message, and the two higher-value tiers pause for a person to approve the return from their inbox.

💡 **Tip:** Recognize the pattern? It's the same tiered-approval idea as Module 1 — where order values above 1,000 and 10,000 USD escalated to a manager and an executive — applied here at return time, and delivered through **Human review** instead of the Approvals connector.

1. On each **shipping-fee branch** from Step 5, add an **M365 Copilot** action to draft the message, configured the same way as **Draft Not Eligible Message** in Step 4 — **Prefer Async** set to **false**. For the **manager** branch (**Less than 50, Greater than 10**), rename the action **Draft Manager Approval Message** and set the message to:

```
A customer wants to return order [Order Number]. Its shipping fee is between $10 and $50, which requires manager approval. Write a short approval request that begins with "Hello," and states the order number and why approval is needed. Output only the final message, ready to send.
```

As in Step 4, replace `[Order Number]` with the **Order Number** value from the **trigger group** in the picker — an approver should read which order they're approving, not a fill-in-the-blank template. Notice these prompts *prescribe* the greeting ("begins with "Hello,") rather than listing what to avoid:

telling the model what to write is more reliable than prohibiting what not to, because vague prohibitions invite it to reason about them inside the output.

The Draft Manager Approval Message action configured with the manager approval prompt

Figure: The first of the three message actions — Draft Manager Approval Message, configured like Step 4's.

2. For the **senior-manager** branch (**Greater than 50**), rename the action **Draft Senior Manager Approval Message** and set the message to:

```
A customer wants to return order [Order Number]. Its shipping fee exceeds $50, which requires senior-manager approval. Write a short approval request that begins with "Hello," and states the order number and why approval is needed. Output only the final message, ready to send.
```

The Draft Senior Manager Approval Message action configured on the senior-manager branch

Figure: **Draft Senior Manager Approval Message** on the **Greater than 50** branch, configured like the manager message.

3. For the **approved** branch (the **Else** of *Manager Approval Validation*, fee below \$10, no approval needed), rename the action **Draft Approved Message** and set the message to:

```
Create a short customer-ready message that the return for order [Order Number] is approved. Write a single ready-to-send message with no placeholders.
```

The Draft Approved Message action configured on the Else (below \$10) branch

Figure: **Draft Approved Message** on the **Else** (below \$10) branch — an approved message with no review.

4. Add a **Human review** action to the **manager** branch (**Less than 50, Greater than 10**). If this is the first time you're using **Human review**, create its connection with the default option when prompted (the same way you created the Dataverse connection in Step 2). Rename the action **Manager Review**, and set the **Title** to:

```
Return Review Needed
```

5. For the **Message**, use the **Response** dynamic value of **Draft Manager Approval Message**. Under **Assigned to (first to respond)**, enter yourself as the reviewer — assigning yourself is a **test-only** shortcut that keeps the lab self-contained; a production solution routes each tier to the real manager or senior manager for the order. Set **Channel** to **Outlook** — the review will arrive as an actionable **Request information** email.

Then, under **Inputs**, choose **Yes/No** — inputs are the fields the reviewer fills in when responding, and the action requires at least one, so the panel flags a validation error until you add it. The input shows a default prompt; replace it with a clear question so the reviewer knows exactly what **Yes** and **No** mean — for example **Approve this return? (Yes = approve, No = decline)**. Set the question on the input itself, not in the message above it.

The Manager Review panel with the message, assignee, Outlook channel, and the Yes/No input added

Figure: The Manager Review panel filled top to bottom — message, assignee, channel, and the required Yes/No input.

6. Repeat for the **senior-manager** branch (**Greater than 50**): add a second **Human review** renamed **Senior Manager Review**, with the title **Return Review Needed**, the **Response** dynamic value of **Draft Senior Manager Approval Message** as the message, yourself under **Assigned to (first to respond)**, the **Outlook** channel, and a **Yes/No** input with the same clear question.

Designer overview of the whole step: the three shipping-fee branches with their message actions and the two Human review approvals

Figure: The whole step at a glance — three drafted messages on the shipping-fee branches, with **Manager Review** and **Senior Manager Review** on the two approval branches.

✓ **Checkpoint:** The two higher-value branches each generate an approver message and request a **Yes/No** approval over email; the **approved** branch generates an approved message.

Step 7: Capture the outcome and complete the response

A review collects a **Yes/No** decision — but collecting it isn't the same as *acting* on it. In this step each approval branch acts on the reviewer's answer and writes a **customer-ready** message (approved or declined) into the **response** variable, so the agent receives a clear outcome rather than a raw **Yes / No**. Every branch still converges on the single **Respond to the agent** action.

1. On the **manager** branch (**Less than 50, Greater than 10**), after **Manager Review**, add an **If/Else** condition and rename it **Act on manager review**. In its left value, use the review's **Yes/No** response (the output of **Manager Review**), and set the condition to check whether it equals **Yes**.

The Act on manager review condition set to check whether the Manager Review response equals Yes

Figure: The **Act on manager review** condition — testing the Manager Review response for **Yes**.

- **If (true — approved):** add a **Variable → Update Variable** action named **Update Manager Approved**, and set **response** to a customer-ready approval, for example:
Good news — your return for order [Order Number] has been approved. Please use the enclosed instructions to send the item back.

The Update Manager Approved action on the If (true) branch writing an approval message into response

Figure: The **If (true)** branch — **Update Manager Approved** writes an approval message into response.

- **Else (false — declined):** add a **Variable → Update Variable** action named **Update Manager Declined**, and set **response** to a customer-ready decline, for example: **After review, we're unable to approve the return for order [Order Number] at this time.**

The Act on manager review condition with Update Manager Approved on the true branch and Update Manager Declined on the false branch

Figure: Both outcomes wired — approved on **If (true)**, declined on **Else (false)** — each writing a customer-ready message into **response**.

2. On the **senior-manager** branch (**Greater than 50**), repeat the same pattern after **Senior Manager Review**: add an **If/Else** on its **Yes/No** response and rename it **Act on senior manager review**, with **Update Senior Manager Approved** on the approved branch and **Update Senior Manager Declined** on the declined branch, each writing a customer-ready message into **response**.

The Act on senior manager review condition with approved and declined update actions on its branches

Figure: **Act on senior manager review** — the same approved/declined outcome pattern on the senior-manager branch.

 **Production note:** This lab handles the two decisions a reviewer makes — approve and decline. A production build would also catch a review that returns **no** decision (unanswered, cancelled, or expired) and set **response** to a pending message rather than leaving it empty — see the asynchronous approval pattern in Step 8.

3. The **approved** branch (fee below \$10) needs no review: add a **Variable → Update Variable** action named **Update Auto Approved**, set **response** to the **Response** dynamic value of **Draft Approved Message**, and connect it between **Draft Approved Message** and the respond action.
4. Drag the **Respond to the agent** action to the end and connect **every** branch to it. Along the way the designer may have spawned extra copies of the respond action — they appear numbered, like **Respond to the agent 2** — delete any you find, and reconnect their branch to the original. This flow deliberately ends in a single respond action: the **response** variable exists precisely so one exit can serve every branch.

Every branch converging on a single Respond to the agent action

Figure: Every branch converges on the single **Respond to the agent** action.

5. Select **Save** and review the flow checker. It may warn that **multiple nodes lead into a single action** — that happens whenever several branches converge on one downstream action, as all your branches do on **Respond to the agent**. Here it's expected and safe, because every branch first writes its own message into the single **response** variable, so the shared exit always returns a defined value. Treat the same warning with more caution elsewhere: when converging branches *don't* all set the shared state, it can signal fragile or unsupported control flow. Confirm each converging branch updates **response** before you move on.

The flow checker warnings about multiple nodes leading into a single action, safe to ignore for this lab

Figure: The flow checker's convergence warnings — expected here because every branch sets response before the shared exit.

 **Checkpoint:** Every branch converges on **Respond to the agent**, and every branch — not-found, not-eligible, both approval tiers (approved *and* declined outcomes), and the approved (below-\$10) branch — writes a customer-ready message into the **response** variable that the respond action returns.

Step 8: Publish and test the workflow

Local testing lets you validate the whole flow — including the human-review approvals — before wiring it to a live agent. If a direct run is blocked with the `AgentTriggerTest.notTestable` error you met in

Step 4, that's expected for an agent-triggered flow — you'll validate the same paths through the agent in Step 9.

1. Select **Publish** again, so your latest changes are in the published version — the run always executes the last *published* flow, so unpublished edits from Steps 5–7 would silently not be part of the test. Before publishing, make sure the **flow checker** is clean, or that every remaining warning is one you understand and expect (like the convergence warning explained in Step 7) — don't publish over warnings you can't account for.
2. Stage your test data before running anything. Two things matter for each test order: its **Order Status** and its **shipping fee**. In freshly seeded Northwind data most orders have an **empty** Order Status, so you'll set a few to **Closed** — but rather than *editing* shipping fees, **use the fees already in the data**: sort or filter the Orders view by **Shipping Fee** to find an order that already sits in each range. Open the **Admin Management App** included in the Northwind Traders solution from **Apps** in the Power Apps portal (check the **Shared with me** tab if it isn't in your default list), and pick these four orders, noting their order numbers:
 - **A not-eligible order** — any order whose status is empty or anything other than **Closed**. An empty status also takes the not-eligible path, because *empty* is “not equal to 3” too.
 - **A closed, mid-fee order** — one whose existing shipping fee is **\$10–\$50**; set its **Order Status** to **Closed**.
 - **A closed, high-fee order** — one whose existing shipping fee is **above \$50**; set its **Order Status** to **Closed**.
 - **A closed, low-fee order** — one whose existing shipping fee is **below \$10**; set its **Order Status** to **Closed**.

Sorting the view by **Shipping Fee** makes the low-, mid-, and high-fee orders easy to spot without changing any fee values — you're only staging the **status**, so the sample data stays otherwise intact (and easier to restore afterward).

3. Test the flow with the **play** button. Enter your **not-eligible** order's number in the **Order trigger input** dialog and run it — it should take the **Order is not closed** branch and return the not-eligible message.
4. Run again with your **closed, mid-fee** order — each run leaves you in the **Activity** tab, so switch back to **Build** and select the **play** button for every rerun. This one traverses the **Else > Less than 50, Greater than 10** (manager) branch and sends a manager approval to your inbox. The assigned reviewer receives an actionable **Request information** email — the generated approver message naming the order and its fee rule, **Yes / No** options, and a **Submit** button. Select **Yes** and **Submit**, and watch the run complete — the decision flows back through the `response` variable and closes the loop.

 **Why the caller can time out — and what production does instead.** A **Human review** can take minutes or hours, but the agent-facing **Respond to the agent** action holds its connection open only briefly. If the reviewer doesn't answer within that short window, the run fails at **Respond to the agent** with `ActionResponseTimedOut` (HTTP 504 to the caller) even though the review itself is still valid. This isn't something to work around by “answering quickly” — it's a sign that a **synchronous** tool call is the wrong shape for a long-running human approval. A production design **doesn't block on the approval**: it returns an immediate *pending-review* response to the agent, completes the approval out of band, and notifies the requester through a supported follow-up channel once the decision is made. This lab keeps the synchronous shape only so you can watch the whole loop end-to-end in one sitting; for testing, have Outlook open and answer as soon as the email arrives.

The Request information approval email with the generated message naming the order, Yes/No options, and a Submit button

Figure: The approval email the reviewer receives — a single generated message naming the order, a Yes/No decision, and Submit.

5. Repeat with your **closed, high-fee** order (above \$50) — the **senior-manager** tier works the same way: the approval email arrives from **Senior Manager Review**, and your prompt **Yes** closes the run.
6. Finally, run your **closed, low-fee** order — it takes the **approved** branch (**Else**, fee below \$10) and returns an approved message with no review needed.

The complete Validate Returns workflow after the full test pass

Figure: The complete Validate Returns workflow.

✓ **Checkpoint:** Each order type traverses the expected path, the **Activity** tab highlights the route taken, and every run completes with a message returned to the agent.

Step 9: Use the workflow in an agent

Finally, you add the published workflow to a Copilot Studio agent as a tool — the same handoff pattern that lets any agent call reliable, deterministic automations mid-conversation.

1. You're done with the flow designer — in the left-hand pane, select **Agents**, then select **New agent**.

The Agents page reached from the left-hand pane, with the New agent button highlighted

Figure: From the flow designer to Agents in the left-hand pane — where the agent is created.

2. Configure the agent: set **Name** to the value below and select **Create**.

Northwind Returns Agent

3. Open the agent's **Tools** section and select **Add a tool**. Switch to the **Workflows** tab, select **Validate Returns**, and add it.

Adding the Validate Returns workflow as a tool on the Northwind Returns Agent

Figure: Adding the Validate Returns workflow as a tool on the Northwind Returns Agent.

Then open the agent's **Instructions** and add guidance that keeps it grounded when the tool doesn't return a usable result — this is what stops the agent from guessing:

Use the Validate Returns tool to check whether an order can be returned, passing the order number the customer provides. Base your answer only on the tool's response. If the tool returns no result, an error, or a pending status, do not guess or infer eligibility: tell the customer the request could not be completed or is pending review, and state the next step. Never say an order does not exist unless the tool's response explicitly says so.

4. Set the agent's primary model to **Claude Sonnet 4.6** so it reliably recognizes the order number in the question. Select **Publish**, then open **Preview** and ask:

💡 **Cost note:** Keeping the workflow on the **When an agent calls the workflow** trigger preserves the no-charge inclusion for Microsoft 365 Copilot-licensed users in production. A different trigger (schedule, manual) would bill the workflow's actions — see [Appendix A](#).

```
can customer return order <your not-eligible order number>
```

5. The agent identifies that it should call the **Validate Returns** tool, passes the order number, runs the flow, and returns the result into the chat — the not-eligible path responds in seconds, so this exchange cleanly demonstrates the whole loop: the agent recognizes the request, calls the tool, and rephrases the flow's grounded answer in its own words.
6. Now try an approval path: ask about your **closed, mid-fee** order, and answer the approval email as soon as it arrives. Because the tool call is synchronous (Step 8), a slow answer makes the tool return without a decision — and this is exactly where the agent instructions earn their keep. A properly-instructed agent reports a **transparent pending or failure** message and the next step; it does **not** infer that the order doesn't exist or improvise an eligibility answer from missing output. If you ever see the agent guess, tighten its instructions — a grounded agent must fail safely, treating "no tool output" as *unknown*, never as *ineligible* or *absent*. (In production, the asynchronous pattern described in the closing section removes this race entirely.)

The agent test chat showing both exchanges: the not-eligible answer for one order, and a completed approval — eligible, a senior manager has approved — for another

Figure: The full loop in the agent's chat — an instant not-eligible answer, then an approval completed mid-conversation and relayed as a decision.

✓ **Checkpoint:** The agent automatically calls the flow, interprets its response, and replies with a human-readable answer — sending an approval to your inbox when required, and failing safely (pending or unable-to-complete) when the tool returns nothing.

💡 **Before production:** Return a **structured** workflow response — separate `status`, `outcome`, and `customerMessage` fields rather than a single untyped string — so the agent can act on the outcome without parsing prose. And test the tool across the full range of edge cases before exposing it to customers: a **nonexistent order**, a **workflow failure**, a **pending or declined approval**, and an **approval timeout**.

Step 10 (Optional): Handle an order that doesn't exist

The core build assumes the order number the agent passes matches a real order. If it doesn't, **List Order Rows** returns zero rows, and every expression that reads "the first row" (the status check in Step 3, and the shipping-fee checks in Step 5) fails on an empty result. This optional step adds a **not-found guard** so the flow returns a clear, grounded answer instead.

You add one condition, right after **Initialize Response** and *before* **Order is not closed**, so the guard protects everything downstream.

1. On the connector between **Initialize Response** and **Order is not closed**, insert an **If/Else** condition and rename it **Order found**. In its left value, choose **Insert expression** and use the Copilot expression builder with a prompt such as:

```
Get the number of rows returned by List Order Rows
```

Set the operator to **is equal to** and the value to **0**.

2. On the **If (true)** branch — no order matched — add a **Variable** action with **Update Variable**, rename it `Update Not Found Response`, and set `response` to a grounded message, for example: `No order was found with that order number. Please check the number and try again.`

Connect this branch to the shared **Respond to the agent** action, so the “not found” case returns through the same exit every branch uses.

3. On the **If (false)** branch — an order matched — connect through to the existing **Order is not closed** condition, so the rest of the flow runs only when an order actually exists.

✓ **Checkpoint:** A nonexistent order number now returns a grounded “not found” message; a real order continues into the status and shipping-fee logic. Test both a **real** and a **nonexistent** order number to confirm each branch.

Step 11 (Optional): Read the shipping fee once into a variable

In Step 5 you insert the **same** shipping-fee expression twice — once in **Senior Manager Approval Validation** (`> 50`) and again in **Manager Approval Validation** (`>= 10`). Repeating the expression works, but it duplicates logic: if the source column ever changes, you’d have to fix it in two places, and each condition re-evaluates the same lookup. This optional step reads the fee **once** into a variable and reuses it.

1. Add a **Variable** action after **List Order Rows** (for example, right after **Initialize Response**). In its panel, under **Initialize Variable**, set **Variable name** to `shippingFee`, and set **Type** to **Float** (or **Number**, whichever your designer offers for decimal currency).
2. Rename the action **Initialize Shipping Fee**. For its **Value**, choose **Insert expression** and generate the shipping-fee lookup with a prompt such as:

```
Get the shipping fee from the first row of List Order Rows
```

This resolves the fee once, from the first returned row.

💡 If you also added the **Order found** guard in Step 10, place **Initialize Shipping Fee** on the *order-exists* path (after the guard), so it only runs when there’s a row to read.

3. In **Senior Manager Approval Validation**, replace the inline fee expression in the left value with the `shippingFee` variable (from the **Variables** group in the picker). Keep the operator **is greater than** and the value **50**.
4. In **Manager Approval Validation**, do the same: use the `shippingFee` variable as the left value, with **is greater than or equal to** and **10**.

✓ **Checkpoint:** Both approval conditions now compare the single `shippingFee` variable instead of re-deriving the fee. The behavior is identical, but the fee is read in one place — easier to read, and a single point of change if the source ever moves. In production, the pieces would harden: reviewers would resolve to the real manager or senior manager for the order rather than yourself; and — most importantly — an approval a person might answer hours later belongs in an **asynchronous pattern**. Rather than hold the agent’s tool call open while a human decides (the synchronous shortcut this lab uses for teaching), a production design returns an immediate *pending-review* response and completes the approval out of band, notifying the requester through a supported follow-up channel once the decision is made. For the latest supported guidance on building and publishing workflows, see the [Copilot Studio workflows documentation](#). Published runs also consume Copilot Studio capacity per action, so you’d monitor usage as the returns desk scales. What wouldn’t change is the shape you built — trusted data in, deterministic branches deciding, generative AI phrasing, and a human signing off where it matters.

🎉 Congratulations — you built a workflow that an agent calls mid-conversation to validate returns, combining Dataverse data, deterministic rules, generative messages, and human approvals into a single conversation turn.

Lab completion and cleanup

You built and tested the workflow end to end. Before you move on, tidy up so the shared environment and sample data stay clean for the next person:

- **Restore the sample data.** Step 8 changed the **Order Status** and **shipping fee** of four Northwind orders. Set those orders back to the values you noted before editing, or reseed the Northwind data from [solutions/README.md](#).
- **Remove test artifacts.** Delete the test agent (**Northwind Returns Agent**) and any unpublished or superseded workflow versions you no longer need, along with any test approvals still sitting in your inbox.
- **Review run history safely.** You can inspect past runs from the workflow's **Activity** tab and run history. When you share or export diagnostics, omit customer details and approval content so private data isn't exposed.

Appendix A: Credit consumption & cost transparency

A workflow runs on Copilot Studio's **standard harness** and bills in **Copilot Credits (CC)** — the common currency across Copilot Studio (this replaced “messages” on 1 September 2025). Every figure here is in **Copilot Credits only**.

 **A note on names:** Microsoft's billing and licensing documentation refers to workflows as **agent flows** — so the billing meter is called “*Agent flow actions*” and the docs quoted below say “*agent flow*.” They're the same thing you build in this module's workflows experience.

 **Rates change — always verify.** The rates below are current as of writing, but Microsoft updates them. For the authoritative figures always check [Billing rates and management](#) and the [Copilot Studio Licensing Guide](#).

One meter: cost is per action

The billing model for a workflow is refreshingly simple. Per Microsoft: “*every action your agent flow executes consumes Copilot Studio capacity*.” There isn't a separate meter for the different step types — **List rows**, an **If/Else** condition, a **variable** update, a **Human review**, the **M365 Copilot** message draft, and **Respond to the agent** each count as **one action**:

Meter	What it covers	Rate
Agent flow actions	Every action the workflow executes — data, logic, AI, human-review, and respond steps alike	13 CC per 100 actions (≈ 0.13 CC/ action)

So a run's cost ≈ (number of actions on the path it takes) × 0.13 CC. The M365 Copilot draft step isn't billed at a special generative rate here — inside a workflow it's simply one more action.

One exception — premium models: if an AI action (like the M365 Copilot draft) runs on a **premium/reasoning** model, that step adds the documented **Text and generative AI tools (premium)** charge of **10 CC per 1,000 tokens** on top of its action cost. On a **standard** model, there's no extra token charge — just the per-action rate.

The big win for this workflow: no charge under an M365 Copilot licence

This lab's workflow uses the **When an agent calls the workflow** trigger. Microsoft's rules zero-rate that case: **"Microsoft 365 Copilot licensed users and test runs (from both the flow designer and the agent's test chat) are not affected"** by capacity consumption. In practice:

- **Every test you run in this lab** — from the workflow designer (Step 8) *and* from the agent's test chat (Step 9) — is **no charge**.
- **In production**, when an **M365 Copilot-licensed** user's agent calls the workflow via the agent trigger, the workflow's actions are **no charge** too.
- **Billing kicks in** only for **non-licensed** users, or if you change the workflow to a **different trigger** (schedule, manual, automated) — those consume the per-action rate above.

Per-path consumption in this lab (when billed)

A run traverses **one** path, so it pays only for that path's actions. The action counts below are **estimates** — but you never have to guess: the run's **Activity** view lists the exact actions it executed, and **actual cost = that count × 0.13 CC**.

Path	Rough actions	Est. cost (when billed)
Not found (Step 10) / Not eligible	~5-7	~0.7-0.9 CC
Approved (below \$10)	~7-9	~0.9-1.2 CC
Manager / Senior tier (adds Human review + act-on-review)	~10-13	~1.3-1.7 CC

Takeaway: each run is a fraction of a credit. Real spend is **volume × actions per run** — a returns desk at thousands of requests a day — and it's **zero** while your agent's users are M365 Copilot-licensed on the agent trigger.

Activities that drive higher usage

- **Longer paths** — more conditions, variable ops, or review steps mean more billable actions.
- **Premium/reasoning models** on AI actions — adds the 10 CC/1,000-token premium charge per such step.
- **Volume of runs** — the dominant factor at scale.
- **Trigger type** — a **non-agent trigger** forfeits the M365 Copilot no-charge inclusion.
- **Retries/failures** — a run that fails partway still bills the actions it executed.

Factors that influence variability

- Which **path** a request takes (approval tiers run more actions than not-eligible).
- **Model tier** on the AI actions (standard vs premium).
- Whether the caller is **M365 Copilot-licensed** (no charge vs billed).
- Human-review **rework** (a re-sent review adds actions).

What customers should monitor (business language)

- **Actions per run** — read straight from the run's **Activity** view; $\times 0.13$ CC = the run's cost when billed.
- **Runs $\times$ actions** — model this before scaling the returns desk.
- **Licence coverage** — keep agent users **M365 Copilot-licensed** so agent-triggered workflows stay no-charge.
- **Trigger type** — keep the production tool on the **agent-call** trigger to preserve the inclusion.
- **Model tier** on AI actions — standard unless a premium model measurably helps.
- **Where to look:** the agent's **Analytics** page and capacity reporting in the **Power Platform admin center**; forecast with the [agent usage estimator](#).

Optimising consumption without reducing business value

- **Deterministic branches over generative classification** — this lab's Step 5 routes fee tiers with **If/Else conditions** instead of the AI **Classify** action. Both are actions, but conditions never risk the premium-token charge and are predictable — a cleaner, cheaper design.
- **Read a value once** — Step 11's optional **shippingFee** variable removes a duplicated lookup, trimming an action per run.
- **Only draft where needed** — the below-\$10 path needs no approver message, so don't add one.
- **Right-size the model** on AI actions — a standard model avoids the premium token meter.
- **Keep users M365 Copilot-licensed** on the agent trigger to stay in the no-charge lane.
- **Forecast, then monitor** with the estimator and Analytics.

 **Learn more:** [Agent flows overview](#) · [Billing rates and management](#) · [Standard harness licensing](#) · [Agent usage estimator](#) · [Copilot Studio Licensing Guide](#)

Appendix B: Ways to test a workflow

A workflow can be exercised at several points, and — usefully — **testing never costs Copilot Credits**: per Microsoft, “*testing an agent flow in the designer or from the agent's test chat doesn't consume Copilot Studio capacity.*”

Where	How	Notes
Designer — Test panel	Save and Publish , then select Test and run Manually or Automatically ; inspect each action's output	You must publish first ; a green checkmark means the run succeeded.
Agent test chat (Step 9 Preview)	Ask the agent a question that makes it call the tool	Exercises the <i>whole</i> loop — the agent's orchestration plus the workflow. No capacity used.
Per branch	Stage inputs that force each path (not-eligible, each fee tier, and — with Step 10 — not-found)	The four-path test intent this lab uses.

 **Why the Run button can fail.** A workflow with the **When an agent calls the workflow** trigger is built to run *from an agent*, so a direct designer run can return `AgentTriggerTest.notTestable`. When that happens, validate those paths through the **agent test chat** in Step 9 instead — it isn't a fault in your build.

Because you **can't publish a workflow that still has errors** (the flow checker blocks it), a clean checker is the gate before any test run.

 [Edit and manage your agent flow](#)

Appendix C: Workflow triggers & the sync-vs-async decision

A workflow starts with a **trigger**, and the type you choose determines *how* it runs, *whether* it can be an agent tool, and *how* it bills.

Trigger	Starts when...	Can be an agent tool?	Billing
When an agent calls the workflow (<i>this lab</i>)	An agent invokes it as a tool	✓ Yes	No charge for M365 Copilot-licensed users
Scheduled	A recurrence you define	✗ No	Per-action rate
Automated / event	Another event fires	✗ No	Per-action rate
Instant / manual	You run it on demand	✗ No	Per-action rate

The 100-second rule (the crux of this lab)

To be usable as an agent tool, a workflow **must respond within a 100-second action limit**, be **published**, and respond **synchronously** — the **Asynchronous response** toggle must be **Off** (under **Networking** in the **Respond to the agent** action's settings). Microsoft's guidance: *"Optimize the flow logic, queries, and the amount of data returned so that a typical run is below this 100-second limit."*

This is exactly why a **human approval** breaks the synchronous shape: a person can't reliably answer inside 100 seconds, so the run fails at **Respond to the agent** (the `ActionResponseTimeout` / HTTP 504 you saw in Step 8). It isn't a bug — a long-running human decision doesn't fit a synchronous tool call.

The production pattern: respond now, approve later

Rather than hold the tool call open, a production design **returns an immediate pending-review response**, runs the approval **out of band**, and **notifies the requester** through a supported follow-up channel once the decision is made. The synchronous build in this lab is a teaching shortcut; the asynchronous pattern is the durable shape.

 [Add an agent flow as a tool](#) · [Agent flows overview](#)

Appendix D: The expression builder & common workflow expressions

Several steps in this workflow use **Insert expression** to compute a value the dynamic-content picker can't offer directly. Copilot's **expression assistant** turns a plain-English prompt into an expression — you don't hand-write it. Here's what this workflow generates, and why.

Where	Plain-English prompt	What it does (conceptually)
Step 3 status check	"first row... order status ID value"	<code>first(<List Order Rows>)</code> <code>['nwind_orderstatusid']</code> — reads the status of the first matched row

Where	Plain-English prompt	What it does (conceptually)
Step 5 fee checks	“shipping fee from the first row”	the same <code>first(...)</code> shape, reading the shipping-fee column
Step 10 not-found guard	“number of rows returned”	<code>length(<List Order Rows output>)</code> — the row count, compared to 0

Why `first(...)`? List rows returns a *collection*; `first()` takes the single top row (you set **Row count = 1**), which avoids wrapping the action in an **Apply to each** loop.

Why the reference looks unusual. The assistant refers to the List rows action by an **internal connector ID**, not its display name — so your expression text differs from anyone else’s. Don’t “tidy” it; confirm it reads the first row and the correct column, then accept it as generated.

 **Tip:** Prefer the **assistant** over hand-writing expressions — describe the value you want, generate it, and validate the result rather than memorizing syntax.

 [Edit and manage your agent flow](#)

Appendix E: The single-variable, single-exit pattern

This workflow uses one deliberate design shape: **every branch writes its message into a single response variable, and every branch converges on one Respond to the agent action**. It’s worth understanding as a reusable pattern.

Why it works

- **One source of truth** — `response` holds “the answer,” whatever path produced it (not found, not eligible, approved, declined, pending).
- **One exit** — a single **Respond to the agent** binds its output to `response` *once*. Add a path later and it just writes the variable; the exit needs no change.
- **Fewer moving parts** — no duplicated respond actions to keep in sync.

The convergence warning (expected here)

The flow checker may warn that **multiple nodes lead into one action**. Here that’s **safe**, because every converging branch sets `response` before the shared exit, so the exit always returns a defined value. Elsewhere, treat it with more care — if converging branches *don’t* all set the shared state, it can signal fragile or unsupported control flow. Confirm each branch updates `response`.

When the pattern doesn’t scale

If different callers need **different-shaped** outputs, a single untyped string strains. The production upgrade is a **structured response** — separate `status`, `outcome`, and `customerMessage` fields — so the agent acts on typed fields instead of parsing prose.

 [Edit and manage your agent flow](#)

Appendix F: Deterministic vs generative decisions in a workflow

A workflow mixes two kinds of decision. Knowing which to reach for keeps it reliable and cheaper.

	Deterministic (If/Else, Switch)	Generative (AI action, e.g. M365 Copilot, Classify)
Best for	Exact rules: numbers, status, dates, thresholds	Language and meaning: drafting text, classifying intent
Predictability	Same input → same output, every time	Can vary; may need examples to steer
Cost	Flat per-action rate	Per-action, plus the premium-token charge on a premium/reasoning model
This lab	Fee tiers (Step 5), status check (Step 3)	Drafting customer/approver messages (Steps 4, 6)

The design rule: let **deterministic branches decide the business outcome** (eligibility, approval tier), and let **generative AI only phrase** the customer-facing wording. That’s why Step 5 uses **If/Else numeric conditions** rather than the AI **Classify** action for the fee thresholds — a currency boundary is an exact rule, and a deterministic comparison is more reliable *and* avoids the premium-token risk.

Agent flows are **deterministic by nature**: “*the same input always produces the same output, making them reliable and predictable.*” Reserve AI actions for **genuinely semantic** work (natural language, ambiguous categorization) — and see the cost angle in [Appendix A](#).

 [Agent flows overview](#)

Appendix G: Human review / approval mechanics

The **Human review** action pauses the workflow for a person to decide, then resumes with their answer. This lab uses it for the manager and senior-manager tiers; the settings you configured were:

Setting	What it does	As set in this lab
Title	The heading the reviewer sees	Return Review Needed
Message	The body — here, the drafted approver message	The Response of the draft action
Assigned to (first to respond)	Who can respond; the first answer wins	Yourself (<i>test-only</i>)
Channel	Where the request is delivered	Outlook (an actionable email)
Inputs	The fields the reviewer fills; at least one is required	A Yes/No with a clear question

Test-only vs production

Assigning **yourself** keeps the lab self-contained. In production, the reviewer resolves to the **real** manager or senior manager for the order — via a lookup, a Dataverse role, or a manager field — not a hard-coded person.

The timing constraint (important)

A Human review can take **minutes or hours**, but an **agent-called** workflow must respond within the **100-second** limit (see [Appendix C](#)). So a synchronous review inside an agent tool is inherently fragile — that's the 504 you met in Step 8. Production moves the approval **out of the synchronous path**: respond "pending" immediately, and complete the approval asynchronously.

Acting on the outcome

Collecting a Yes/No isn't enough — **branch on it** (Step 7) so **Yes** yields an approved message and **No** a declined one, and treat **no response** (empty or expired) as *pending*, never as approved.



[Add an agent flow as a tool](#) · [Agent flows overview](#)

Recommended next step

You've built the customer-facing side: an agent that answers shoppers with grounded, approved decisions. Continue to [Module 4: CUA](#), where a second agent — the Northwind Operations Agent — takes on the operations side, using computer use to operate a website no connector or API can reach.